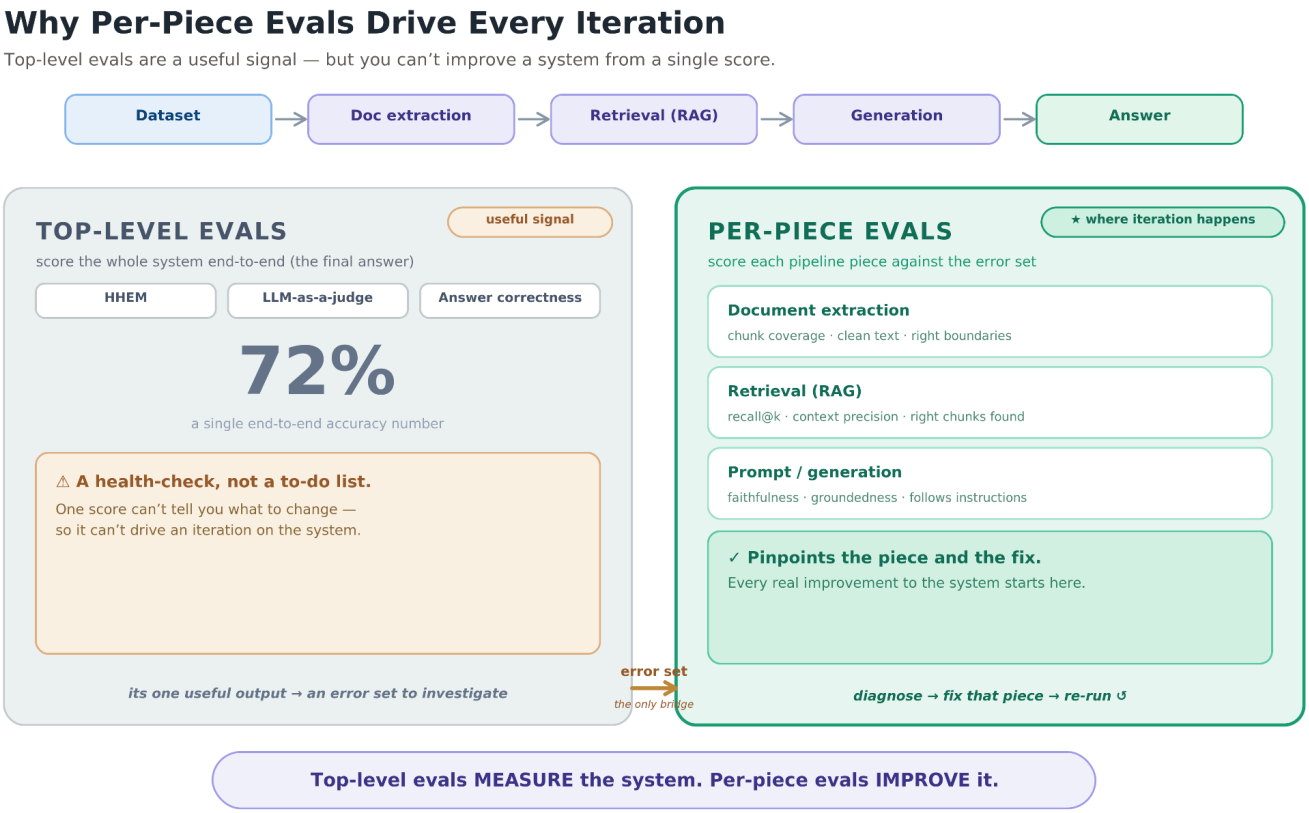
Evaluation Setup Roadmap



Please read this to understand retrieval in agentic systems (guide)

Goals:

The goal of this system is to create an evaluation framework that we can use to improve the TDA system over time in an easier and more efficient way than shooting in the dark. Evals will serve a few key purposes:

- Better understanding of production runs and the data behind the results
- Easier to iterate on specific pieces of the pipeline due to those metrics
- Easier to bucket issues seen in the error set to isolate specific pieces of the pipeline to fix, and then validate the fix not only solves that bucket, but does not worsen other pieces.
- Deterministic metrics to quantify issues

Evaluation Scope:

The evaluation framework should combine four types of measurement:

1. Deterministic evaluation
   Used when we have ground truth, such as gold chunks, expected document spans, expected answers, or expected tool calls. Examples include Recall@K, Full Recall@K, Precision@K, MRR, and valid-tool-call rate.
2. Model-based evaluation
   Used for quality questions that cannot be measured with a simple rule. Examples include whether retrieved content is useful, whether an answer is grounded in evidence, whether an answer is correct, and whether an answer fully addressed a user request.
3. Agent-trajectory evaluation
   Used to evaluate the sequence of search, planning, tool-call, reranking, and stopping actions taken by the system.
4. Human evaluation and calibration
   Used to validate automated evaluation results, investigate unclear cases, and ensure that the scoring system reflects what the business considers a successful answer.

The objective is not to depend on one score. A successful TDA response should retrieve the required evidence, choose the correct tool or search strategy, use retrieved evidence correctly, and produce an answer that is supported, correct, relevant, and complete.

Stakeholder items:

These four components provide the basic structure for the evaluation system:

1. Golden Dataset (contains the test queries and expected results).
2. Retrieval Golden Dataset (identifies the correct chunks for each query).
3. Arize AI (used to track and evaluate system runs and metrics).
4. Hugging Face (can be used later for answer level evaluation such as HHEM).

Arize Phoenix Components:

The following Arize Phoenix capabilities should be added to the evaluation design:

5. **Arize Phoenix Datasets**
   Used to store evaluation examples, production examples, expected answers, gold document spans, metadata, and query categories. Phoenix datasets can be used to run repeatable experiments and evaluations across pipeline versions.

6. **Arize Phoenix Experiments**
   Used to run the same dataset against multiple versions of the TDA pipeline. This makes it possible to compare changes to retrieval, prompts, embedding models, re rankers, answering models, or agent logic.
7. **OpenInference Tracing**
   Used to trace the full path of one user request through the system. OpenInference is designed for AI observability and works with OpenTelemetry conventions.
8. **Arize Phoenix Retrieval Relevance Evaluator**
   Used to determine whether retrieved information is relevant and useful for answering the query. This complements deterministic Recall@K metrics.
9. **Arize Phoenix Faithfulness Evaluator**
   Used to determine whether the final answer is supported by the retrieved context.
10. **Arize Phoenix Correctness Evaluator**
    Used to determine whether the final answer is correct relative to the user request and, when available, a reference answer.
11. **Arize Phoenix Tool Selection Evaluator**
    Used to determine whether the agent selected the appropriate search system, database, retrieval method, or tool.
12. **Arize Phoenix Tool Invocation Evaluator**
    Used to determine whether the agent called a tool correctly with valid arguments, correct formatting, and safe values.
13. **Arize Phoenix Tool Response Handling Evaluator**
    Used to determine whether the agent correctly understood and used the information returned by a tool.
14. **Arize Phoenix Completeness and Conciseness Evaluators**
    Used to determine whether all parts of the user's request were completed and whether the answer included unnecessary repetition or filler.

Phoenix documents evaluators for faithfulness, correctness, retrieval relevance, conciseness, tool selection, tool invocation, and tool-response handling.

Phase 1

The first goal is to set up retrieval tracking for the different search types. We need to understand the system's ability to correctly retrieve the required chunks for a specific query. I'm a believer that the accuracy of a RAG system can never outperform the system's ability to show the correct information to the answering step. Our ability to improve the retrieval will define the tools accuracy. Everything gets easier when you know you have a high probability to retrieve the correct info.

Phase 1 Scope:

Phase 1 should remain focused on retrieval recall. We should not begin by optimizing the final answer model before we understand whether the answering model receives the correct evidence.

The core Phase 1 question is:

> *Did the system retrieve all of the evidence needed to answer the question correctly?*

The secondary Phase 1 question is:

> *Was the retrieved information useful and relevant to the question, even when the exact gold chunk did not appear?*

This means Phase 1 should use both:

- **Deterministic retrieval metrics,** including Recall@K, Full Recall@K, Precision@K, MRR, and Gold Chunk Rank.
- **Arize Retrieval Relevance**, which evaluates whether the retrieved information actually helps answer the user's request.

Recall@K alone is important but incomplete. A result can contain a gold chunk but still include too much irrelevant context, rank the relevant content too low, or fail to give the final answering model enough usable information.

Requirements:

- Dataset of queries and correct chunks in a given set of docs
- Harness to run queries on the docs and evaluate % of the time the correct chunk is in the set handed to answering models
- Scoring at different top-k levels
- For search system c: tracking of rounds
- Capture the final chunks provided to the answering model after all filtering and reranking.
- Capture raw retrieval results before reranking so retrieval failures can be separated from reranking failures.
- Capture the search query used in every round, including any query rewrites generated by the agent.
- Capture the search system or tool used for every retrieval action.
- Capture latency, token use, and estimated cost per search round.
- Capture duplicate chunks and duplicate searches.
- Capture empty retrieval responses and tool errors.
- Run Retrieval Relevance evaluation on final retrieved context and, where useful, on each retrieval round.

- Store all retrieval traces and evaluation results in Arize Phoenix.

NOTE: We should work with the stakeholders to create this dataset if possible. We just need an understanding of the documents that correlate to a query to answer.

Implementation Steps:

The first phase should focus only on checking whether the retrieval system is finding the correct information.

- Create the Retrieval Golden Dataset.
- Connect each test query with its correct chunk or chunks.
- Run the same queries through the different search systems.
- Record the chunks returned by each search system.
- Record the ranking of each returned chunk.
- Compare the returned chunks with the correct chunks.
- Calculate Recall@K for each search system.
- Track how many rounds Search System C requires.
- Create a stable `query_id` for every evaluation query.
- Label correct evidence using document IDs and text spans rather than only chunk IDs.
- Add a `query_type` label, such as factual lookup, multi-document, comparison, ambiguous, multi-step, or insufficient-information.
- Add an `expected_tool` field when a specific search system or tool should be selected.
- Run the same dataset through each candidate pipeline version.
- Store each run as an Arize Phoenix Experiment.
- Log pipeline version, prompt version, embedding model, reranker version, top-k value, and search configuration.
- Capture the raw retrieval result and the final reranked result.
- Calculate Recall@1, Recall@3, Recall@5, Recall@10, Full Recall@K, Precision@K, MRR, and Gold Chunk Rank.
- Run Retrieval Relevance on the retrieved context.
- Compare results by search system, query type, number of required chunks, document count, and search-round count.
- Review failed examples using traces rather than relying only on aggregate scores.

Set up retrieval recall:

We create a dataset of queries to correct chunks inside a document set. We should cover good data variance of:

- Queries
- Chunks
- Number of chunks needed for a correct answer

- Number of documents in the set
- Number of rounds search system c takes to finalize
- Answer complexity
- Single-document questions.
- Multi-document questions.
- Questions requiring multiple chunks from the same document.
- Questions requiring several independent facts.
- Questions requiring a comparison between sources.
- Questions that require structured filtering, such as date, category, entity, status, or record type.
- Ambiguous questions that require clarification.
- Questions where no supporting information exists in the available documents.
- Questions that should result in “insufficient information” rather than a generated answer.
- Questions known to have failed in production.
- Queries with different levels of wording variation.
- Queries requiring different search strategies.
- Queries requiring a single search round.
- Queries requiring several search rounds.
- Queries where the most relevant document is not the most obvious lexical match.

Dataset Setup:

The Retrieval Golden Dataset should contain enough variation to represent different types of real queries.

The dataset should include:

- Different query types.
- Different relevant chunks.
- Questions requiring one or multiple chunks.
- Different document-set sizes.
- Simple and complex questions.
- Queries requiring different numbers of search rounds.

This will prevent the evaluation from being based on only one type of question or document.

Dataset Fields:

The Retrieval Golden Dataset should contain the following fields:

Field	Description
Query ID	Stable ID used to compare results across experiments

Query	The user question or request
Query Type	Lookup, comparison, multi-document, ambiguous, multi-step, and so on
Gold Document IDs	Documents containing required evidence
Gold Text Spans	Exact source passages required to answer correctly
Required Chunk Count	Number of evidence chunks required
Reference Answer	Expected answer, when applicable
Expected Tool	Expected search system or tool, when applicable
Difficulty	Simple, medium, complex, or multi-step
Expected Outcome	Answer, clarification, refusal, or insufficient information
Query Category	Business or product category used for analysis
Notes	Labeling details and edge cases

The use of text spans is important. Chunking strategies may change over time, but source document spans remain stable. Labeling only chunk IDs would make the dataset fragile when chunk size, overlap, or splitting rules change.

Dataset Definition:

The dataset should also define:

- Source of queries: production, synthetic, or both (to be confirmed).
- Who labels the correct chunks: to be confirmed.
- Target size, such as 100–300 queries to start.
- Gold answers should be labeled by document and text span rather than chunk ID, so changes to chunking do not break the dataset.
- Who approves final gold labels.
- How disagreements between labels are resolved.
- How often new production failures are added.
- Which query categories are considered high priority.
- Which categories are release-blocking.

- Whether the dataset is divided into development, validation, and holdout test sets.
- Which examples are used for prompt iteration and which examples are held back for unbiased validation.
- Which examples require human review because they are ambiguous or business-sensitive.

A recommended starting dataset size is 100–300 queries. However, the target is not only volume. The initial dataset should include representative coverage of the most important TDA workflows and known failure modes.

Evaluation Harness:

We set up the system to run these queries against the different datasets and create a ranking of the chunks that come back.

We set up the harness to create a % score at each top-k level. For runs where top-k is set to more than 1, we still want to know the % at top-k one.

Harness Setup:

The evaluation harness will automate the testing process.

For every evaluation query, it should:

- Send the query to the search system.
- Collect the returned chunks.
- Store the ranking of those chunks.
- Compare them against the correct chunks.
- Calculate the retrieval score.
- Repeat this for different top-k values.
- Store the results for comparison between runs.
- Record the raw retrieval output before reranking.
- Record the reranked output.
- Record the exact final context provided to the answer model.
- Record the search query used for every round.
- Record whether the query was rewritten by the system.
- Record the search system or tool selected.
- Record tool arguments.
- Record tool responses.
- Record all search rounds.
- Record when the correct chunk was first retrieved.
- Record the final answer.
- Record latency for each search step and total latency.

- Record token usage and estimated cost when applicable.
- Record tool errors and system failures.
- Run deterministic retrieval metrics.
- Run Arize Retrieval Relevance evaluation.
- Store the output as an Arize Phoenix Experiment result.
- Store the trace ID so each failure can be investigated step by step.

The harness should be able to compare:

- Search System A versus Search System B versus Search System C.
- Different embedding models.
- Different chunking strategies.
- Different top-k values.
- Different reranking strategies.
- Different query rewrite prompts.
- Different agent stopping rules.
- Different answer-generation prompts.
- Different answering models.

This makes the evaluation repeatable instead of manually checking individual queries.

Arize AI Setup:

Arize AI should be used to track evaluation runs, datasets, experiments, traces, configurations, and evaluation metrics.

The setup should include:

- OpenInference tracing with one span per search call and one child span per round for Search System C.
- Uploading the Golden Dataset as an Arize Dataset.
- Running each pipeline version as an Arize Experiment to compare runs.
- Logging the pipeline version and configuration with every run.

Why OpenInference Tracing Is Important:

A trace represents the full path of a single request through the TDA system. Individual spans represent the operations inside that request, such as retrieval, query rewriting, reranking, generation, or tool calls.

For this project, tracing should make it possible to inspect the complete execution path for a failed query:

1. Original user query.
2. Query rewrite, if applicable.
3. Selected search system or tool.
4. Search round one.
5. Chunks retrieved in round one.
6. Search round two, if needed.
7. Chunks retrieved in round two.
8. Reranker output.
9. Final context sent to the answer model.
10. Final answer.
11. Deterministic metric results.
12. Model-based evaluator results.

This is important because a poor final answer can come from several different causes:

- The correct evidence was never retrieved.
- The evidence was retrieved but ranked too low.
- The evidence was removed by reranking.
- The agent selected the wrong tool.
- The agent created a poor search query.
- The agent stopped too early.
- The final model ignored retrieved evidence.
- The final model added unsupported content.

Without traces, these failure types are difficult to separate.

Capturing the following for each evaluation run:

- Query
- Retrieved chunks
- Chunk ranking
- Search rounds
- Final answer
- Evaluation metrics
- Evaluation result
- Query ID.
- Query type.
- Pipeline version.
- Prompt version.
- Search system.
- Search query used in each round.

- Embedding model.
- Reranker version.
- Top-k value.
- Document collection or source.
- Tool name.
- Tool arguments.
- Tool response.
- Final context sent to the answer model.
- HHEM score, when enabled.
- Faithfulness score, when enabled.
- QA Correctness score, when enabled.
- Retrieval Relevance score.
- Tool Selection score.
- Tool Invocation score.
- Tool Response Handling score.
- Latency.
- Token use.
- Estimated cost.
- Error status.
- Trace ID.
- Experiment ID.

Agentic Evaluation:

The evaluation should also check how the agent behaves during the retrieval process.

It should evaluate:

- Performance of each search system on the same query.
- Whether Search System C stopped at the right time once it had all required chunks.
- Whether the agent repeated the same searches unnecessarily.
- Recall after each search round, not only the final total round count.

Run-to-Run Consistency:

Agent runs are not deterministic. The same query can take a different search path, use a different number of rounds, and produce a different final answer on each run, so a single pass over the golden dataset can overstate how reliably the system answers.

- Run each golden-dataset query several times per pipeline version. Report both the single-run pass rate and pass^k, which the τ-bench paper defines as the chance that all k independent runs of a task succeed, averaged across tasks. There is no standard number of runs; it is a trade-off between cost and statistical precision.
- In τ-bench's retail tasks, GPT-4o succeeded on about 61% of tasks in a single run, but on under 25% when all 8 runs had to succeed (pass^8). Single-run accuracy can hide this inconsistency.
- For Search System C, also record how much the round count, search path, latency, and token use vary across runs of the same query.

Agent Tool Evaluation:

For an agentic system, retrieval quality alone is not enough. The agent must also select the correct tool, create correct tool arguments, understand the results, and stop at the right time.

Arize Phoenix provides evaluators for Tool Selection, Tool Invocation, and Tool Response Handling. These cover three different stages of the agent tool-calling process.

Evaluator	**Main Question**	**Application in TDA**
Tool Selection	Did the agent choose the appropriate tool?	Did the agent select the best search system, data source, or retrieval method?
Tool Invocation	Did the agent call the tool correctly?	Did it provide correct query text, filters, document scope, date range, and other parameters?
Tool Response Handling	Did the agent use the returned data correctly?	Did it extract the right facts, handle errors, and avoid unsupported conclusions?

Tool Selection Evaluation:

Tool Selection should answer:

> *Given the user query and available tools, did the agent choose the correct search system or tool?*

Examples of failures include:

- Using a semantic search tool when structured filtering is required.
- Searching the wrong document collection.
- Selecting a general retrieval process when a specialized tool is needed.
- Starting multi-round research when a direct document lookup would be sufficient.
- Selecting a tool that cannot access the required source.

Tool Invocation Evaluation:

Tool Invocation should answer:

> *Did the agent generate a correct, valid, and safe tool call?*

This should evaluate:

- Correct tool name.
- Correct search query.
- Correct filters.
- Correct document scope.
- Correct entity or date parameters.
- Required arguments present.
- Proper format.
- Safe argument values.
- No unnecessary tool calls.

Tool Response Handling Evaluation:

Tool Response Handling should answer:

> *Did the agent correctly understand the tool output and use it appropriately in the next step and final answer?*

This should evaluate:

- Whether the agent extracted the correct evidence.
- Whether it ignored relevant results.
- Whether it misinterpreted records or documents.
- Whether it handled empty responses correctly.
- Whether it handled tool errors safely.
- Whether the final answer is supported by the tool output.
- Whether it continued searching unnecessarily after finding enough evidence.

Retrieval Metrics:

The following metrics can be used to measure retrieval performance:

Metric	What It Measures	Why It Is Useful
Recall@1	Share of required chunks found in the top 1 result.	Shows first-result retrieval quality
Recall@3	Share of required chunks found in the top 3 results	Measures short-list retrieval quality
Recall@5	Share of required chunks found in the top 5 results	Measures broader retrieval coverage
Recall@10	Share of required chunks found in the top 10 results	Measures retrieval coverage at a larger context
Gold Chunk Rank	Rank of each required chunk (worst rank reported for multi-chunk queries)	Shows how high the relevant information was ranked
Search Round Count	Number of rounds required by Search System C	Shows how many steps the agent needed
Per-Query Recall	Recall score for a single query	Helps identify failed queries

Category Recall	Recall grouped by query/category type	Helps identify specific areas with weaker retrieval
Precision@K	Measures how many of the retrieved top-K results are relevant	Shows whether retrieval contains unnecessary results
MRR	Measures how highly the first relevant result is ranked	Shows whether the relevant chunk is buried in the results
Full Recall@K	Whether every required chunk is in top K	Shows whether the answer model had all required evidence
Retrieval Relevance	Whether retrieved information is useful for the user's query	Adds semantic retrieval quality beyond exact gold matching
Duplicate Chunk Rate	Frequency of duplicate chunks in results	Identifies wasted context and poor retrieval diversity
Empty Retrieval Rate	Frequency of no useful returned content	Identifies weak search queries or retrieval gaps
Round-by-Round Recall	Recall after each agent round	Shows when the evidence was found
Duplicate Search	Frequency of repeated or	Identifies inefficient agent

Rate	near-identical searches	behavior
Stop Condition Success	Whether agent stopped after gathering enough evidence	Measures agent efficiency
Tool Selection Accuracy	Whether the selected tool was appropriate	Measures planning quality
Tool Invocation Validity	Whether tool calls had correct, safe arguments	Measures execution quality
Tool Response Handling Accuracy	Whether output was understood and used correctly	Measures reasoning over retrieved data
Retrieval Latency	Time taken to retrieve useful evidence	Measures user experience
Token Usage and Cost	Resource use per query or run	Measures operating efficiency

Metric Definitions:

The evaluation should clearly define how metrics behave when a query requires multiple chunks.

- Recall@K: Number of required chunks found in the top K, divided by total required chunks. A query needing 3 chunks with 2 found scores 0.67.
- Full Recall@K: 1 if all required chunks are in the top K, otherwise 0. This shows whether the answering model had everything it needed.

- Recall@1 limit: For queries needing more than one chunk, Recall@1 cannot exceed 1 divided by the number of required chunks, so results should be reported separately for single-chunk and multi-chunk queries.
- Gold Chunk Rank: For multi-chunk queries, record the rank of every required chunk and report the worst (lowest) rank, since the answer is only complete once all chunks are retrieved.
- Retrieval Relevance: Measures whether the retrieved information is relevant and useful for answering the user query. This is different from Recall@K because a chunk can be relevant even if it was not specifically labeled as the gold chunk, and a gold chunk can be present but not sufficient on its own.
- Faithfulness: Measures whether the final answer is grounded in the retrieved context. It answers whether the answer contains claims that are supported by the evidence supplied to the model.
- QA Correctness: Measures whether the final answer correctly answers the user's question, using retrieved evidence and a reference answer when available.
- Answer Relevance: Measures whether the final answer directly addresses the user's request instead of including unrelated or generic information.
- Completeness: Measures whether every active part of the user's request was completed. This is important for multi-part questions and tasks requiring several outputs.
- Tool Selection Accuracy: Measures whether the agent selected the appropriate tool or search system for the task.
- Tool Invocation Validity: Measures whether the tool call had correct arguments, correct formatting, and safe values.
- Tool Response Handling Accuracy: Measures whether the agent interpreted and used tool output correctly.

Scoring Scope:

Recall should be measured on the final set of chunks provided to the answering model after reranking, rather than only on the raw search results.

Raw Retrieval Versus Final Context:

The evaluation should calculate retrieval metrics at two points:

1. **Raw retrieval output**
   This shows whether the search system found relevant evidence at all.
2. **Final context provided to the answer model**
   This shows whether relevant evidence remained after filtering, deduplication, reranking, truncation, and context-window limits.

This distinction is important.

If the relevant chunk appears in raw retrieval but disappears from the final context, the problem is likely in reranking or context construction rather than retrieval.

If the relevant chunk appears in the final context but the answer is incorrect, the problem is likely in answer generation, prompt design, model behavior, or evidence use.

Round Tracking:

For Search System C, each evaluation run should record how many search rounds were required before the system finalized its retrieved information.

The evaluation should record:

- Query ID
- Number of search rounds
- Chunks retrieved in each round
- Round in which the correct chunk was found
- Final retrieved chunks
- Search query issued by the agent.
- Search system or tool selected.
- Tool arguments.
- Retrieval result.
- Retrieval relevance score.
- Recall achieved after that round.
- Whether the agent had already retrieved all required evidence.
- Whether the agent continued searching after sufficient evidence was found.
- Whether the agent repeated a previous search.
- Whether the tool returned an error or empty response.
- Time and token usage for the round.

This will help determine whether the system is finding the required information immediately or needs multiple search attempts.

A useful analysis question is:

> *In which round did the system first retrieve all required evidence, and how many additional rounds did it take after that point?*

This will show whether Search System C is stopping too early, stopping too late, or repeating searches unnecessarily.

Phase 2 - Retrieval Recall Metrics

We set up a few deterministic metrics that I believe will make iterating on the results shown in recall easier to adjust. This is mostly focused on saying “we see specific query types that have issues, how can we quantify those issues”.

Implementation:

After the initial Recall@K evaluation is working, the next step is to break down the results to understand where retrieval problems are occurring.

The results can be analyzed by:

- Query type
- Search type
- Number of required chunks
- Number of documents
- Search rounds
- Answer complexity
- Search system.
- Data source or document collection.
- Chunking strategy.
- Chunk size.
- Chunk overlap.
- Embedding model.
- Reranker version.
- Top-k value.
- Query rewrite version.
- Prompt version.
- Agent stopping-rule version.
- Expected tool.
- Actual selected tool.
- Tool error status.
- Query ambiguity.
- Single-chunk versus multi-chunk query.
- Single-document versus multi-document query.
- Whether the query required structured filtering.
- Whether the query required multiple rounds.
- Production versus synthetic query source.

This will help identify whether a low recall score is caused by a specific type of query or by a broader retrieval issue.

Failure Bucket Analysis:

Each failure should be placed into a specific error bucket where possible:

- Correct evidence was not retrieved.
- Correct evidence was retrieved but ranked too low.
- Correct evidence was removed by reranking.
- Too much irrelevant context was retrieved.
- The query rewrite was poor.
- The agent selected the wrong tool.
- The agent invoked the tool incorrectly.
- The agent repeated unnecessary searches.
- The agent stopped before all evidence was found.
- The agent continued searching after evidence was already sufficient.
- The answer did not use available evidence.
- The answer contained unsupported information.
- The answer was factually incorrect.
- The answer was incomplete.
- The tool result was misinterpreted.
- Tool error was not handled correctly.

These error buckets allow the team to prioritize fixes based on the actual source of failure rather than only looking at an aggregate score.

Phase 3 - Top-Level Answer Metrics

Phases 1 and 2 measure whether the system retrieves the right evidence. Phase 3 measures the outcome the user sees. These top-level metrics are the numbers reported to stakeholders and used for release decisions. The retrieval and tool metrics remain diagnostic: they explain why a top-level metric moved.

The core Phase 3 question is:

> *Did the user get a correct, supported, and complete answer, or an appropriate "insufficient information" response?*

Headline Metrics:

Each headline metric is scored pass/fail for every query. The evaluator sections that follow (HHEM, Faithfulness, QA Correctness, Answer Relevance, and Completeness) describe the individual measurements.

Metric	Question	Measured With
Answer Correctness	Is the answer right?	Phoenix QA Correctness judge, compared with the Reference Answer
Groundedness	Is every claim supported by the final context sent to the answer model?	Phoenix Faithfulness judge, plus HHEM or Granite Guardian as a second check
Fact-Level Completeness	Are all required facts in the answer?	Strict vital recall on reviewed nuggets (see Fact-Level Completeness)
Appropriate Abstention	Did the system decline when, and only when, the context was insufficient?	Sufficient-context label compared with whether the system answered, checked against the Expected Outcome field

Answer Success Rate is the composite headline number: the share of queries that pass every headline check that applies to them. For a query whose Expected Outcome is insufficient information, only Appropriate Abstention applies. The four component metrics show why the composite moved.

Guardrail Metrics:

Guardrail metrics are tracked alongside the headline metrics but are not optimized directly. A pipeline change should not make them worse.

- Latency per query.
- Token use and estimated cost per query.
- pass^k across repeated runs of the same query (see Run-to-Run Consistency).

Diagnostic Metrics:

Diagnostic metrics explain a change in a headline metric rather than define success. They include Answer Relevance, Conciseness, the Phase 1 and Phase 2 retrieval metrics, and the tool evaluation metrics.

Phase 3 Requirements:

- Calibrate before reporting. Before a headline judge's scores are reported, compare the judge with human labels on a held-out sample. Husain and Shankar recommend 100 to

200 labeled examples per failure mode, and measuring both the true positive rate (how many real failures the judge catches) and the true negative rate (how many good answers it correctly passes). Arize AX's align-evals workflow reports accuracy, a confusion matrix, and per-label precision and recall, and Arize advises at least 50 to 100 labeled examples before trusting the metrics. This moves part of the human review and calibration step from the end of the roadmap to the start of Phase 3.

- Oracle baseline. Run the answer model with only the gold evidence as its context. This sets the ceiling for answer quality. The gap between the oracle run and the full pipeline is the loss caused by retrieval; failures in the oracle run itself come from generation.
- No-retrieval baseline. Run the answer model with no retrieved context. Queries it answers correctly from the model's own knowledge inflate scores without testing the document collection, so flag them in the dataset.
- Failure attribution. Join each failed answer with its Full Recall@K on the final context. If required evidence was missing, attribute the failure to retrieval. If the evidence was present but the answer still failed, attribute it to generation, as described in Raw Retrieval Versus Final Context.
- Reporting. Break results out by query type and search system. Compare each pipeline version query by query against a pinned baseline experiment, and report a confidence interval with every headline number. This follows Anthropic's guidance to report standard errors and paired differences when comparing versions.

Phase 3 Exit Criteria:

- Each headline judge has been calibrated against human labels.
- Baseline numbers are recorded for the current pipeline, the oracle run, and the no-retrieval run.
- Pass thresholds for release-blocking query categories are agreed with stakeholders.

HHEM/HELM:

https://huggingface.co/vectara/hallucination_evaluation_model

While I think this mostly going to be used in future evals, it can be helpful to review the rate of similarity between our answer and chunks.

Future Evaluation:

HHEM can be added after the retrieval evaluation is established.The purpose would be to evaluate whether the generated answer is supported by the retrieved chunks.

This creates the next level of evaluation:

Retrieval Evaluation → Did we retrieve the correct information?

Answer Evaluation → Did the final answer correctly use that information?

HHEM Role in the Evaluation Framework:

HHEM should be used as an answer-support or hallucination-risk signal.

The key question is:

> *Does the generated answer contain claims that are unsupported by the retrieved context?*

For each answer evaluated with HHEM, store:

- Query.
- Retrieved context.
- Final answer.
- HHEM score.
- Pipeline version.
- Prompt version.
- Search system.
- Number of search rounds.
- Query category.
- Retrieval Recall@K.
- Full Recall@K.
- Retrieval Relevance score.
- Human-review outcome, when available.

HHEM should be reported by:

- Overall score.
- Low-support answer rate.
- Query type.
- Search system.
- Search-round count.
- Retrieval-quality bucket.
- Pipeline version.
- Prompt version.

HHEM Limitations:

HHEM should not be used as the only answer-quality metric.

HHEM does not replace:

- Recall@K.
- Full Recall@K.
- Retrieval Relevance.
- QA Correctness.
- Answer Relevance.
- Completeness.
- Tool Selection.
- Tool Invocation.
- Tool Response Handling.

An answer can be grounded in retrieved context but still be incomplete, irrelevant, or incorrect for the user's question. Likewise, an answer can appear good even when retrieval misses an important source.

Phoenix Faithfulness Evaluation:

Phoenix Faithfulness should be added alongside HHEM.

The Faithfulness evaluator evaluates whether an answer is supported by the provided context. It is used to detect unsupported or ungrounded claims in generated responses.

Faithfulness should answer:

> Is the final answer grounded in the retrieved context passed to the model?

Recommended inputs:

- User query.
- Retrieved context.
- Final answer.

Recommended outputs:

- Faithful or unfaithful label.
- Score.
- Explanation.
- Trace ID.
- Query ID.

Faithfulness is particularly useful for identifying cases where retrieval was strong but the answer-generation model added unsupported detail, misinterpreted the retrieved evidence, or ignored relevant context.

Phoenix QA Correctness Evaluation:

Phoenix QA Correctness should be used after retrieval evaluation is stable.

QA Correctness should answer:

> *Did the system correctly answer the user's question based on the retrieved data?*

Recommended inputs:

- User query.
- Retrieved context.
- Final answer.
- Reference answer, when available.

QA Correctness should be used to identify:

- Answers that are factually wrong.
- Answers that answer the wrong question.
- Answers that confuse entities or dates.
- Answers that provide only partial information.
- Answers that should have stated that information was insufficient.
- Answers that are grounded in context but still not correct.

Phoenix Answer Relevance and Completeness:

Answer Relevance:

Answer Relevance should answer:

> Does the final answer directly address the user's question?

This is useful for detecting answers that are technically related to the topic but do not actually complete the requested task.

Completeness:

Completeness should answer:

> *Did the response complete every active part of the user's request?*

Completeness is important for:

- Multi-part questions.
- Requests involving several documents.
- Comparison questions.
- Requests asking for a result plus explanation.
- Requests asking for several items, records, or fields.
- Agent workflows that must complete multiple steps.

Phoenix includes a Completeness evaluator intended to assess whether every active user request was completed.

Fact-Level Completeness (Nugget Recall):

The Phoenix Completeness evaluator checks whether each active user request was completed. It does not check whether every required fact made it into the answer, so an answer can address every part of a request and still leave out key information.

- The TREC 2025 RAG track measures this by breaking each question into short required facts (nuggets). LLMs draft the nuggets and NIST assessors can edit them. Each nugget is labeled vital (indispensable to a full answer) or okay (helpful but non-essential).
- The track's main metric, strict vital recall, is the share of vital nuggets that the answer fully supports. In the track's manually assessed results, the highest-scoring RAG run reached only 0.39 (Table 7).
- For TDA: draft nuggets for each golden-dataset query with an LLM, have a reviewer edit them, and store them with the Reference Answer as part of the dataset version. Report strict vital recall by query type, especially for multi-part, comparison, and multi-document queries.

Tool Evaluation Models:

Phoenix has prebuilt evaluators for Tool Selection, Tool Invocation, and Tool Response Handling.

Phoenix Evaluator	**Input Fields**	**Output Labels**	**Use for This Project**
Faithfulness Evaluator	Input, output, context	Faithful / unfaithful	Checks whether final answer is supported by retrieved context
Correctness	Input, output	Correct / incorrect	Checks whether final

Evaluator			answer correctly responds to the query
Retrieval Relevance Evaluator	Input, context	Relevant / irrelevant	Checks whether retrieved information is useful
Conciseness Evaluator	Input, output	Concise / verbose / too concise	Checks unnecessary verbosity after quality is stable
Tool Selection Evaluator	Input, output, tool name, expected tool	Correct / incorrect	Checks whether the right search tool was chosen
Tool Invocation Evaluator	Input, tool name, tool call arguments	Correct / incorrect	Checks whether tool calls contain valid parameters
Tool Response Handling Evaluator	Input, tool response, output	Good / bad	Checks whether tool results were interpreted correctly

Evaluator Independence:

LLM judges should stay independent of the system they grade. If a judge shares a model, prompt, or reference material with the pipeline, the evaluation can reward the pipeline for matching the judge rather than for being correct.

- Use a different model, ideally from a different model family, for the Retrieval Relevance judge than for Search System C's reranker. Clarke and Dietz simulated systems that use the same LLM relevance assessor (UMBRELA) as their final-stage reranker and found that this distorts system rankings. By extension, a judge that matches System C's reranker is likely to favor System C over Systems A and B.
- Keep judge prompts, gold nuggets, and reference answers out of the prompt-tuning loop. Dietz et al. showed that a RAG system can reach near-perfect evaluation scores when elements of the evaluation, such as prompt templates or gold nuggets, are leaked or can be predicted, and they recommend blind evaluation settings.
- In practice: store judge prompts and gold references separately from the pipeline's prompts, do not expose them to the people or agents tuning the pipeline, and reserve the holdout split described in Dataset Definition for final validation.

Supplementary Open-Source Tooling:

Arize AX remains the system of record for datasets, experiments, traces, and evaluation results. The tools below fill specific gaps in Arize's built-in evaluators. They run as evaluators in experiments launched from the Arize SDK (client.experiments.run). These evaluators execute in the team's own Python environment and log their scores back to Arize AX, so all results stay in one place.

ranx: Retrieval Metrics and Significance Testing

Open source, MIT license: https://github.com/AmenRa/ranx

- Arize's built-in code evaluators (Exact Match, Matches Regex, JSON Parsable, and keyword checks) do not compute retrieval ranking metrics. ranx computes Recall@K, Precision@K, MRR, nDCG, MAP, Hit Rate, and R-Precision, and its results are checked against trec_eval, the standard information-retrieval scoring tool. The Phase 1 deterministic metrics do not need to be hand-coded.
- The evaluation compares three search systems on 100–300 queries. ranx's paired t-test, Fisher's randomization test, and Tukey's HSD test show whether a difference between Search Systems A, B, and C, or between two pipeline versions, is real or within noise.
- Integration: map the gold document spans to the chunk IDs produced by the pipeline version under test and load them into ranx as relevance judgments, then load each experiment's ranked chunks as a run. Log per-query scores back to Arize as experiment evaluations and store the significance-test results with the experiment comparison.

IBM Granite Guardian 4.1: Second Grounding Model

Open weights, Apache-2.0 license, released April 2026:
https://huggingface.co/ibm-granite/granite-guardian-4.1-8b

- Vectara's own published comparison shows the open HHEM model (HHEM-2.1-Open) dropping to 62.6% balanced accuracy on long inputs, compared with 78.2% for Vectara's commercial HHEM-2.3. Search System C's multi-round context produces this kind of long input, so HHEM should not be the only grounding signal.
- Granite Guardian checks groundedness, context relevance, answer relevance, and hallucinated tool calls. Arize's pre-built judges cover document relevance (Relevance Evaluator) and answer correctness (QA Evaluator), but not whether the answer addresses the question, so Granite Guardian also supplies the Answer Relevance metric. Its tool-call check supports the Tool Invocation evaluation.
- Deployment: an 8B-parameter model that needs a GPU. It is served with vLLM and called from a Python evaluator in Arize experiments. It supports English only. IBM reports 0.760–0.764 balanced accuracy on the LLM-AggreFact grounding benchmark.
- Lighter alternative: LettuceDetect (open source, MIT license) highlights the exact unsupported text spans in an answer instead of returning one score per answer: https://github.com/KRLabsOrg/LettuceDetect

Google Sufficient Context Autorater: Context Sufficiency and Abstention

Published prompt, free to use. The prompt is in Appendix C.1 of the paper: https://arxiv.org/abs/2411.06037

- The autorater labels whether the retrieved context is sufficient to answer the question. It does not need a gold answer to produce this label (Google Research blog). In the paper, Gemini 1.5 Pro with a one-shot prompt classified sufficiency with 93% accuracy on 115 expert-labeled examples (Table 1).
- It tests the insufficient-information bucket directly. Comparing the sufficiency label with whether the system answered or abstained shows when the system answered anyway. In Google's tests, Gemma answered incorrectly on 10.2% of questions when given no context at all, and on 66.1% when given insufficient context (Google Research blog). Insufficient context made wrong answers more likely than no context.
- Run it after each Search System C round as well as on the final context. The share of queries that move from insufficient to sufficient after an additional round shows whether extra rounds are worth their latency and cost. It runs as a custom LLM-as-a-judge evaluator in Arize AX experiments.

Overall Evaluation Flow:

This evaluation workflow measures retrieval, answer quality, and tool performance. It compares results against a baseline to identify failures and improve the pipeline.

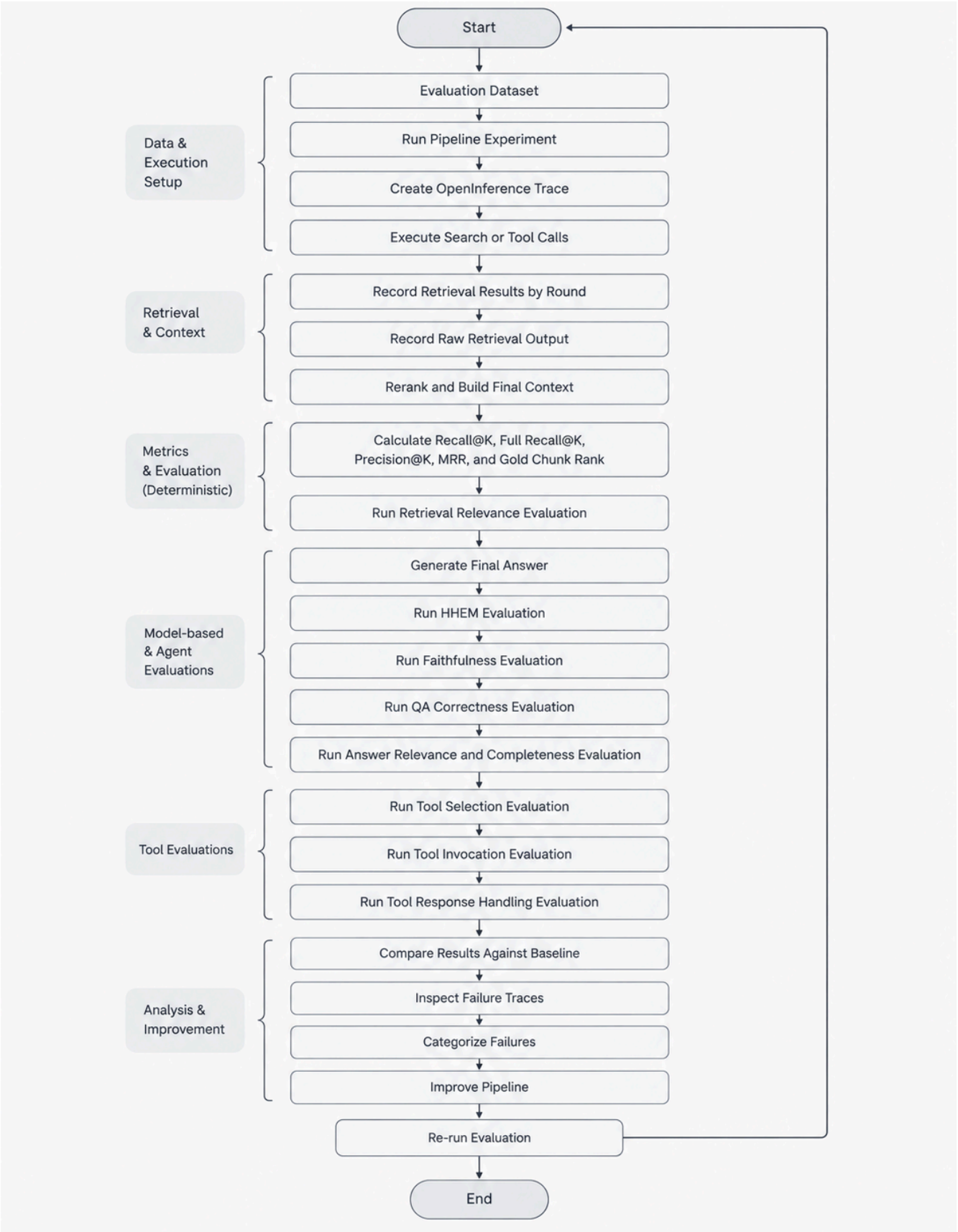

Start
Data & Execution Setup
Evaluation Dataset
Run Pipeline Experiment
Create OpenInference Trace
Execute Search or Tool Calls
Retrieval & Context
Record Retrieval Results by Round
Record Raw Retrieval Output
Rerank and Build Final Context
Metrics & Evaluation (Deterministic)
Calculate Recall@K, Full Recall@K, Precision@K, MRR, and Gold Chunk Rank
Run Retrieval Relevance Evaluation
Model-based & Agent Evaluations
Generate Final Answer
Run HHEM Evaluation
Run Faithfulness Evaluation
Run QA Correctness Evaluation
Run Answer Relevance and Completeness Evaluation
Tool Evaluations
Run Tool Selection Evaluation
Run Tool Invocation Evaluation
Run Tool Response Handling Evaluation
Analysis & Improvement
Compare Results Against Baseline
Inspect Failure Traces
Categorize Failures
Improve Pipeline
Re-run Evaluation
End

Figure X. End-to-End Evaluation Workflow

The process should be repeatable so that each pipeline change can be evaluated against the same dataset and metrics.

Expected Result:

After the full evaluation framework is established, the team should be able to answer the following questions for any pipeline version:

- Did the system retrieve the correct evidence?
- Did it retrieve all required evidence for multi-chunk questions?
- Was the retrieved evidence relevant and useful?
- Was relevant evidence ranked highly enough?
- Did the agent choose the correct search system or tool?
- Did the agent invoke the tool correctly?
- Did the agent understand and use tool results correctly?
- Did the agent stop searching when sufficient evidence was available?
- Did the final answer remain grounded in retrieved evidence?
- Did the final answer answer the user's actual question?
- Did the final answer complete every part of the request?
- Did a system change improve the target metric without causing regressions?
- Did quality improve without unacceptable increases in latency, token use, cost, or tool errors?

The immediate priority remains retrieval evaluation. The recommended implementation order is:

1. Retrieval Golden Dataset.
2. Deterministic retrieval metrics.
3. Arize Phoenix tracing.
4. Arize Experiments for pipeline comparison.
5. Retrieval Relevance evaluation.
6. HHEM and Faithfulness evaluation.
7. QA Correctness, Answer Relevance, and Completeness evaluation.
8. Tool Selection, Tool Invocation, and Tool Response Handling evaluation.
9. Human review and calibration of automated evaluators.

References:

- *Arize Phoenix Documentation. Arize AI.* https://arize.com/docs/phoenix
- *Phoenix Datasets and Experiments Overview. Arize AI.* https://arize.com/docs/phoenix/datasets-and-experiments/overview-datasets
- *Build an Evaluation Harness in Phoenix. Arize AI.* https://arize.com/docs/phoenix/cookbook/datasets-and-experiments/building-your-own-eval-harness
- *OpenInference Documentation. Arize AI.* https://arize.com/docs/phoenix/resources/openinference
- *Phoenix Evaluation Documentation. Arize AI.* https://arize.com/docs/phoenix/evaluation/llm-evals
- *Phoenix Prebuilt Evaluation Metrics. Arize AI.* https://arize.com/docs/phoenix/evaluation/pre-built-metrics
- *Phoenix Retrieval Relevance Evaluator. Arize AI.* https://arize.com/docs/phoenix/evaluation/pre-built-metrics/retrieval-relevance
- *Phoenix Faithfulness Evaluator. Arize AI.* https://arize.com/docs/phoenix/evaluation/pre-built-metrics/faithfulness
- *Phoenix Correctness Evaluator. Arize AI.* https://arize.com/docs/phoenix/evaluation/pre-built-metrics/correctness
- *Phoenix Completeness Evaluator. Arize AI.* https://arize.com/docs/phoenix/evaluation/pre-built-metrics/completeness
- *Phoenix Tool Selection Evaluator. Arize AI.* https://arize.com/docs/phoenix/evaluation/pre-built-metrics/tool-selection
- *Phoenix Tool Invocation Evaluator. Arize AI.* https://arize.com/docs/phoenix/evaluation/pre-built-metrics/tool-invocation
- *Phoenix Tool Response Handling Evaluator. Arize AI.* https://arize.com/docs/phoenix/evaluation/pre-built-metrics/tool-response-handling
- *HHEM: Hallucination Evaluation Model. Vectara, Hugging Face.* https://huggingface.co/vectara/hallucination_evaluation_model
- *Phoenix LLM-as-a-Judge Documentation. Arize AI.* https://arize.com/docs/phoenix/evaluation/concepts-evals/llm-as-a-judge
- *ranx: Ranking Evaluation Library. GitHub.* https://github.com/AmenRa/ranx
- *Granite Guardian 4.1 8B. IBM, Hugging Face.* https://huggingface.co/ibm-granite/granite-guardian-4.1-8b
- *LettuceDetect. KRLabs, GitHub.* https://github.com/KRLabsOrg/LettuceDetect

- *Hallucination Detection: Commercial vs. Open Source. Vectara.* https://www.vectara.com/blog/hallucination-detection-commercial-vs-open-source-a-deep-dive
- *Arize AX: Run Experiments in Code. Arize AI.* https://arize.com/docs/ax/improve/experiment-in-code
- *Arize AX: Code Evaluations. Arize AI.* https://phoenix.arize.com/docs/ax/evaluate/evaluators/code-evaluations
- *Arize AX: Pre-built LLM-as-a-Judge Evaluators. Arize AI.* https://arize.com/docs/ax/evaluate/llm-as-a-judge/arize-evaluators-llm-as-a-judge
- *Sufficient Context: A New Lens on Retrieval Augmented Generation Systems. Joren et al., arXiv:2411.06037.* https://arxiv.org/abs/2411.06037
- *Deeper Insights into Retrieval Augmented Generation: The Role of Sufficient Context. Google Research.* https://research.google/blog/deeper-insights-into-retrieval-augmented-generation-the-role-of-sufficient-context/
- *LLM-based relevance assessment still can't replace human relevance assessment. Clarke and Dietz, arXiv:2412.17156.* https://arxiv.org/abs/2412.17156
- *Insider Knowledge: How Much Can RAG Systems Gain from Evaluation Secrets? Dietz et al., arXiv:2601.13227.* https://arxiv.org/abs/2601.13227
- *Overview of the TREC 2025 Retrieval Augmented Generation (RAG) Track. Upadhyay et al., arXiv:2603.09891.* https://arxiv.org/abs/2603.09891
- *τ-bench: A Benchmark for Tool-Agent-User Interaction in Real-World Domains. Yao et al., arXiv:2406.12045.* https://arxiv.org/abs/2406.12045
- *AI Evals: Everything You Need to Know. Hamel Husain and Shreya Shankar.* https://hamel.dev/blog/posts/evals-faq/
- *Arize AX: Align Evals to Human Feedback. Arize AI.* https://arize.com/docs/ax/evaluate/align-evals-to-human-feedback
- *A Statistical Approach to Model Evaluations. Anthropic.* https://www.anthropic.com/research/statistical-approach-to-model-evals